

Professores:

Celso Rodrigo Giusti **The Best**

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

Consumo de API

REST



Consumo de API

Hoje iremos abordar: como o app mobile, especificamente, se conecta a essa API e lida com tudo que pode dar errado no caminho.

- Backend já visto: como a API funciona por dentro;
- Hoje: como o app consome essa API de fora;
- Diferencial mobile: rede instável, tela pequena, usuário esperando resposta.

O app é o cliente HTTP

No mobile, quem faz a requisição não é o navegador nem o Postman, é o próprio código do app, rodando no celular do usuário

- fetch (nativo) ou axios (biblioteca) fazem o papel do "cliente HTTP";
- A URL da API precisa ser alcançável pelo celular/emulador
- Toda requisição no mobile é assíncrona, a tela não pode travar esperando resposta

Promise

Uma Promise é um objeto do JavaScript que representa o resultado futuro de uma operação assíncrona, algo que ainda não terminou, mas vai terminar em algum momento.

- Resolve problemas de código assíncrono (que não trava a execução esperando resposta);
- `.then()` executa quando a Promise é cumprida (sucesso);
- `.catch()` executa quando a Promise é rejeitada (erro).

Fetch – Nativo

fetch() é a forma nativa do navegador de fazer requisições HTTP. Ela retorna uma Promise que resolve assim que os headers da resposta chegam, não o conteúdo já pronto.

- Nativo do browser, não precisa instalar nada;
- Retorna uma Promise com um objeto Response (não os dados ainda);
- Só cai no .catch() em erro de rede, erros HTTP (404, 500) não disparam catch automaticamente

Response e Data

O objeto **Response** é um envelope: traz status, headers, url, mas o corpo ainda vem como **stream bruto**, não interpretado.

`res.json()` lê esse **stream** e converte pra objeto JavaScript.

- **Response** → informações sobre a resposta (`res.status`, `res.ok`, `res.headers`);
- **`res.json()`** → lê e converte o corpo;
- **`data`** → o conteúdo já pronto pra usar (array, objeto, etc.)

Stream e Chunks

Um Stream é uma forma de receber dados aos poucos, em pedaços (chunks), em vez de esperar tudo chegar de uma vez só.

O corpo (body) de uma Response do fetch é um **ReadableStream**: uma sequência desses pedaços de bytes brutos, ainda não decodificados como um texto.

```
chunk 1: <Uint8Array 5b 7b 22 69 64 22 3a ...>  
chunk 2: <Uint8Array 31 2c 22 6e 6f 6d 65 ...>  
chunk 3: ...
```

axios faz a mesma requisição HTTP, mas já entrega os dados prontos em `res.data`, sem exigir o passo extra de `.json()`. Além disso ele trata erros HTTP automaticamente.

- Biblioteca externa (precisa instalar: `npm install axios`);
- JSON já vem parseado em `res.data`;
- Cai no `.catch()` também para erros HTTP (404, 500), não só de rede;
- Tem recursos prontos: interceptors, timeout, cancelamento;

Fetch vs Axios: Requisições HTTP em JavaScript

FETCH

```
fetch('/api/usuarios')  
  .then(res => res.json())  
  .then(data => {  
    setUsuarios(data);  
  })  
  .catch(err => console.error(err));  
  
// converte a resposta com res.json()
```

AXIOS

```
axios.get('/api/usuarios')  
  .then(res => {  
    setUsuarios(res.data);  
  })  
  .catch(err => console.error(err));  
  
// já entrega os dados em res.data
```

A mesma requisição com async/await

FETCH

```
async function getUsuarios() {  
  try {  
    const res = await fetch('/api/usuarios');  
    const data = await res.json();  
    setUsuarios(data);  
  } catch (err) {  
    console.error(err);  
  }  
}  
  
// dois awaits: resposta + res.json()
```

AXIOS

```
async function getUsuarios() {  
  try {  
    const res = await axios.get('/api/usuarios');  
    setUsuarios(res.data);  
  } catch (err) {  
    console.error(err);  
  }  
}  
  
// um await: dados em res.data
```

Métodos HTTP

- GET – Usado sempre que o app precisa carregar uma lista ou um item da API para exibir na tela;
- POST – Usado quando o app precisa criar algo novo na API;
- PUT – Usado para alterar algo que já existe na API;
- DELETE – Usado para remover um recurso da API a partir do seu identificador.

JSON é como os dados trafegam entre API e app, é o "**idioma**" que os dois falam.

- Objetos e arrays em texto: { "nome": "Ana", "idade": 22 }
- JavaScript converte JSON em objeto automaticamente (response.json())

XML foi o formato mais comum antes do JSON dominar. Vale reconhecer, mas não é o que vocês vão usar no dia a dia mobile

- Mais verboso: tags de abertura/fechamento para tudo;
- Ainda aparece em integrações legadas (alguns bancos, notas fiscais, SOAP);
- Regra prática: se a API for nova, é JSON.

Estados de Tela

Toda tela que consome API precisa saber se está carregando, se deu certo ou se deu errado, e mostrar isso pro usuário.

- loading: spinner enquanto espera resposta;
- erro: mensagem clara, sem travar o app;
- sucesso: dado renderizado normalmente;

Conexão Instável é a REGRA

No mobile o usuário está no ônibus, no elevador, com wifi ruim — o app precisa se comportar bem mesmo assim.

- Nunca assumir que a requisição vai dar certo
- Timeout e erro de rede são cenários normais, não raros
- Boa UX de erro é o que separa um app amador de um profissional

Consumo de API

Atividade



A nossa atividade será:

CRIAR UM APP PARA CONSUMIR A API DO SABOR-DIGITAL.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>